جامعة دمياط — كلية الحاسبات والذكاء الاصطناعي (2025/2026)

🥗 NutriAI — دليل الأسئلة والأجوبة المتوقعة لمناقشة التخرج

المشروع: NutriAI: An Intelligent Personalized Diet Recommendation System

إشراف: أ.د. أحمد محمد ربيع | **إعداد:** يوسف علي، أحمد إبراهيم، أحمد محمد، عبد الوهاب أحمد

س 1: ليه استخدمتوا Large Language Models (LLMs) ومستخدمتوش Machine Learning عادي أو Rule-Based Expert Systems؟

🎯 الهدف من السؤال: اختبار فهم المبرر الهندسي والمعماري لاختيار التقنية وليس مجرد اتباع الموضة.

الأنظمة التقليدية من نوع `Rule-Based Systems` تعتمد على شجرات قرارات وقواعد مسبقة وثابتة. هذه الأنظمة تفشل تماماً عند مواجهة معضلة **التعقيد التراكمي المتداخل (Combinatorial Complexity)**؛ فمثلاً مريض يعاني من سكري نوع ثانٍ + ارتفاع ضغط دم + حساسية جلوتين + يفضل نظاماً نباتياً + يتناول دواء Metformin في نفس الوقت. تغطية كل هذه الاحتمالات برمجياً يحتاج ملايين شروط `if-else` المستحيلة واقعياً.

أما نماذج الـ Machine Learning التقليدية (كالـ Classification)، فوظيفتها تصنيفية وليست توليدية؛ لا يمكنها كتابة خطة سردية متكاملة لـ 15 يوماً بـ 60 وجبة مع مقاديرها وملاحظاتها.

استخدامنا لنماذج الـ LLM (خاصة Llama-3.3-70B) وفر لنا **Deep Semantic Understanding** لفهم العلاقات المعقدة بين الأمراض والمغذيات، وتوليد وجبات متنوعة ومتوازنة، مع القدرة على تتبع سياق المحادثة في الشات لاستبدال الأطعمة لحظياً.

س 2: نماذج الـ LLMs معروف إنها بتعاني من الـ Hallucination (الهلوسة).. إزاي تضمنوا الأمان الإكلينيكي لمريض سكر أو ضغط؟

🎯 الهدف من السؤال: مناقشة الأمان الإكلينيكي (Clinical Safety Guardrails) ومنع التوصيات الخطرة.

لم نترك الموديل يولد بحرية عشوائية، بل أحطناه بـ 4 طبقات أمان هندسية صارمة:

- **Low Temperature (0.2):** ضبطنا الـ Temperature على 0.2 في توليد الخطط لتقليل الـ Entropy وإجبار الموديل على الالتزام الصارم بالـ Tokens الأكثر حتمية وموثوقية طبية.
- **Strict Clinical System Prompt:** حقن الموديل بقواعد إكلينيكية إلزامية غير قابلة للتفاوض: منع السكريات البسيطة للسكري، تقليل الصوديوم (<1500mg) للضغط، ومنع مسببات الحساسية بشكل قطعي.
- **Structured JSON Output Contract:** إجبار الموديل على إرجاع مخرجاته داخل JSON Schema محددة ومسبقة الفحص تحتوي على حقل إلزامي اسمه `medical_adjustments` يوثق كل تعديل علاجي قام به الموديل.
- **Post-Generation Programmatic Scan:** فحص برمجي للرد في الباك إند للتأكد من خلوه من أي مكون محظور قبل عرضه، مع إرفاق Disclaimer قانوني يوضح حدود النظام كأداة إرشادية.

س 3: ليه مقسمين المشروع Backend (FastAPI) و Frontend (Streamlit)؟ ما كان ممكن يتعمل كله في ملف Streamlit واحد؟

🎯 الهدف من السؤال: اختبار فهم مبادئ هندسة البرمجيات (System Architecture & Separation of Concerns).

اعتمدنا مبدأ الفصل المعماري (Architectural Decoupling) لسببين جوهريين:

- **Scalability & Reusability:** بناء الباك إند بواسطة FastAPI يتيح RESTful API مستقل وموثق بالكامل بـ Swagger/OpenAPI. هذا يعني أنه لو قررنا غداً بناء تطبيق موبايل (Flutter أو React Native)، لن نعيد كتابة سطر واحد في الباك إند أو الذكاء الاصطناعي، بل سيربط تطبيق الموبايل مباشرة مع نفس الـ Endpoints.
- **Asynchronous Concurrency:** خادم FastAPI مبني على معيار ASGI ومكتبة `asyncio`، مما يمكنه من معالجة آلاف الطلبات المتزامنة (Non-blocking I/O) دون تجميد الخادم، بينما Streamlit يعيد تنفيذ الكود كاملاً (Script Re-run) عند كل تفاعل للمستخدم، فدمج العمليات الثقيلة داخله كان سيسبب بطئاً وتجمداً في الواجهة.

س 4: الموديل بيشتغل بـ Groq و OpenRouter.. إيه الفرق الجوهري بينهم وليه الاتنين معاً؟

🎯 الهدف من السؤال: اختبار فهم البنية التحتية والاستدلال السحابي وتجنب الـ Single Point of Failure.

صممنا النظام كـ **Multi-Provider Architecture** لتحقيق أعلى موثوقية:

- **Groq:** يعتمد على عتاد مخصص يسمى LPU (Language Processing Unit) بدلاً من معالجات GPU التقليدية، مما يحقق سرعة استدلال مذهلة تصل إلى 250-300 توكن/ثانية. نستخدم عليه موديل Llama-3.3-70B Versatile لتوليد خطة الـ 15 يوماً كاملة (أكثر من 5,600 توكن) في أقل من 20 ثانية فقط!
- **OpenRouter:** يعمل كـ Unified Gateway تتيح الوصول لأكثر من 100 موديل مختلف (مثل GPT-4o-mini و Mistral) كبديل مرن واقتصادي.
- **الفائدة الهندسية:** تفادي الـ Single Point of Failure؛ فإذا حدث انقطاع أو تجاوز لمعدل الطلبات (Rate Limit) في Groq، يمكن للمستخدم بضغطة زر من القائمة الجانبية التحويل لـ OpenRouter فوراً دون إعادة تشغيل السيرفر.

س 5: لو انقطع الإنترنت تماماً، هل النظام يتوقف عن العمل؟ وما هو محرك الـ Offline Engine؟

🎯 الهدف من السؤال: اختبار مرونة النظام والعمل في ظروف عدم توفر الشبكة (Offline Resilience).

النظام مصمم ليعمل 100% بدون إنترنت عبر **Offline Clinical Engine** مدمج في الباك إند:

- يعتمد على معادلات التغذية الأيضية المعيارية عالمياً (Mifflin-St Jeor Equation) لحساب السعرات ومعدل الأيض الأساسي BMR والاحتياج الفعلي TDEE.
- يطبق خوارزميات إكلينيكية محددة لتوزيع الماكروز ومطابقة الوجبات المناسبة لكل مرض وحساسية بصورة حتمية وآمنة طبياً.
- يولد خطة الـ 15 يوماً والماكروز في أجزاء من الثانية محلياً بدون أي اتصال بالإنترنت وبدون مفاتيح API، ويعمل تلقائياً كبديل فوري (Graceful Fallback) عند انقطاع الشبكة.

س 6: كيف تم حساب السعرات والماكروز رياضياً وعلمياً في النظام؟

🎯 الهدف من السؤال: اختبار الأساس الرياضي والطبي المعتمد في المشروع.

اعتمدنا معادلة **Mifflin-St Jeor**، وهي الأكثر دقة والمعتمدة لدى الجمعية الأمريكية للتغذية:

1. **حساب معدل الأيض الأساسي (BMR):**
   للذكور: (10 × الوزن كجم) + (6.25 × الطول سم) - (5 × العمر) + 5
   للإناث: (10 × الوزن كجم) + (6.25 × الطول سم) - (5 × العمر) - 161
2. **حساب الاحتياج اليومي الفعلي (TDEE):** ضرب BMR في معامل النشاط الحركي (من 1.2 للخامل إلى 1.9 لشديد النشاط).
3. **تعديل السعرات حسب الهدف:** عجز 450-500 سعرة لإنقاص الوزن، فائض 300-350 سعرة لبناء العضلات، ومطابقة للثبات.

س 7: في الكتاب والـ ERD مكتوب قاعدة بيانات، ولكن في الكود الحالي التخزين In-Memory.. ما هو التبرير الهندسي؟

🎯 الهدف من السؤال: فخ كلاسيكي في مناقشات التخرج لاختبار أمانة واحترافية الطالب في توضيح الـ Trade-offs.

في مرحلة الـ Academic Prototype، كان الهدف الهندسي هو التحقق من خط أنابيب الذكاء الاصطناعي (AI Prompt & Validation Pipeline) والأمان الإكلينيكي.
استخدام الـ In-Memory Dictionaries وفر لنا استجابة فائقة للـ Endpoints بأقل من 3 مللي ثانية، مع حماية بيانات المرضى من التسريب على الأقراص الصلبة أثناء الاختبارات.
ومع ذلك، قمنا برسم مخطط الـ ERD بالكامل في الفصل الثالث ليمثل المستهدف المستقبلي لقاعدة بيانات PostgreSQL الإنتاجية، ووثقنا في الفصل الخامس خطة الانتقال لربط الداتابيز وتشفير كلمات المرور بـ Bcrypt.

س 8: ما هو الدور الحقيقي لمكتبة Pydantic في المشروع؟ وهل هي مجرد Type Hinting؟

🎯 الهدف من السؤال: اختبار المهارات التكنيكال في معايير كود بايثون وهندسة الـ API.

الـ Pydantic في مشروعنا هو **حارس البوابة الأول (First Line of Defense)**:

- يقوم بفرض قيود صارمة وقت التشغيل (Runtime Data Validation)؛ فيمنع قبول عمر خارج نطاق 10-100 سنة، أو وزن خارج نطاق 30-300 كجم.
- يرفض الطلبات غير المطابقة فوراً بكود HTTP 422 مع ذكر الحقل غير الصالح بدقة، مما يمنع إرسال أي بيانات مشوهة للموديل ويوفر التكلفة واستهلاك التوكنز.
- تستخدم مكتبة `pydantic-settings` لإدارة ملف الإعدادات `.env` بطريقة Type-Safe تفشل فوراً عند الإقلاع (Fail-Fast) إذا كان هناك إعداد ناقص، بدلاً من انهيار السيرفر فجأة في الإنتاج.

س 9: كيف تضمنون إرجاع JSON سليم من الـ LLM دون أن يفسد الماركداون أو النصوص التوضيحية الواجهة؟

🎯 الهدف من السؤال: اختبار الـ Defensive Programming و الـ Post-Processing.

بنينا طبقة معالجة دفاعية داخل دالة `parse_llm_json()` :

- تقوم بإزالة علامات الكود `` `json `` و `` ` `` تلقائياً إذا أضافها الموديل.
- تصلح الأقواس المزدوجة الناتجة عن التنسيق `{{` و `}}` .
- تستخدم دالتي `find('{')` و `rfind('}')` لقص واستخراج كائن الـ JSON فقط وتجاهل أي نص شارح قبله أو بعده.
- تتحقق من وجود المفاتيح الإلزامية مثل `meal_plan` وترفض أي كائن ناقص برمجياً قبل تمريره للواجهة.

س 10: لماذا تم ضبط الـ Temperature عند 0.2 لتوليد الخطة وعند 0.7 لشات المتابعة؟

🎯 الهدف من السؤال: فهم الـ Hyperparameters وتأثيرها على استقرار وموثوقية مخرجات الذكاء الاصطناعي.

الـ Temperature يتحكم في اتساع أو حدة توزيع الاحتمالات (Probability Distribution):

- **في توليد الدايت (0.2):** المهمة طبية صارمة تتطلب الالتزام الدقيق بالقواعد الإكلينيكية وهيكل الـ JSON، فالقيمة المنخفضة تجعل الموديل شبه حتمي (Deterministic) وعالي الموثوقية.
- **في الشات (0.7):** المهمة حوارية وتفاعلية، فالقيمة الأعلى تمنح الموديل مرونة لغوية وتنوعاً في صياغة النصائح وبدائل الأكلات مع بقائه مقيداً بسياق الخطة المعتمدة مسبقاً.

س 11: كيف تم تأمين واجهة الـ API ضد اختراق خصوصية المستخدمين؟

🎯 الهدف من السؤال: اختبار أمان التطبيقات والمصادقة (API Security & Access Control).

يعتمد النظام على معيار **JWT (JSON Web Tokens)** المشفر بخوارزمية **HS256**:

- كل مسار محمي يتطلب تمرير الـ Token في ترويسة `Authorization: Bearer` ويتم فحصه عبر Dependency Injection في FastAPI.
- **حماية الجلسات (Session Scoping):** حتى لو امتلك المهاجم توكن صالح، بمجرد محاولته طلب جلسة تخص مستخدماً آخر، يتحقق السيرفر من تطابق اسم المالك مع اسم المستخدم المشفر في التوكن، ويرد بـ **HTTP 404** عمداً لمنع تسريب أي معلومات تفيد بوجود الجلسة أصلاً.

س 12: ما هي التكلفة الحقيقية لتشغيل هذا النظام على أرض الواقع؟

🎯 الهدف من السؤال: دراسة الجدوى الاقتصادية واستهلاك الموارد (Cloud Economics).

تعتبر التكلفة شبه منعدمة مقارنة بالاستضافة الذاتية:

- استضافة موديل 70B محلياً تتطلب كارتي شاشة من فئة A100 بذاكرة 80GB VRAM بتكلفة آلاف الدولارات شهرياً.
- عبر الـ API Inference من Groq، توليد خطة 15 يوماً كاملة يستهلك حوالي 6,700 توكن إجمالاً، بتكلفة تبلغ حوالي **0.004 دولار فقط** (أقل من نصف سنت أمريكي لكل خطة)، مما يحقق أعلى جدوى اقتصادية وقابلية للتوسع التجاري.

س 13: ما هي خريطة الطريق المستقبلية لنقل المشروع من نموذج أولي إلى منتج طبي معتمد؟

🎯 الهدف من السؤال: استعراض رؤية الفريق للتطوير المستمر وأخلاقيات الذكاء الاصطناعي الطبي.

تتضمن خارطة الطريق في الفصل الخامس 4 ركائز أساسية:

1. **RAG Architecture:** ربط النظام بقواعد بيانات وزارة الزراعة الأمريكية (USDA FoodData Central) لحساب السعرات من بيانات مخبرية حقيقية بدلاً من تقديرات الموديل.
2. **Dietitian-in-the-Loop:** لوحة تحكم تسمح لأخصائي التغذية المعتمدين بمراجعة وتعديل الخطط قبل اعتمادها للمريض.
3. **Wearables Integration:** ربط التطبيق بالساعات الذكية (Apple Health / Fitbit) لتحديث معدل النشاط الحركي الفعلي وإعادة ضبط السعرات أسبوعياً.
4. **Mobile PWA & Arabic Localization:** تعريب الواجهة بالكامل وتضمين أسماء الأكلات الشعبية المصرية والعربية.
